

```
1.  /**
2.   * @file Componente para seleccionar aspectos cognitivos.
3.   * @description Permite a los usuarios seleccionar diferentes aspectos cognit
4.   * @requires react
5.   * @requires prop-types
6.   * @requires ../atoms/Slider
7.   * @requires ../atoms/Checkbox
8.   * @requires ../../styles/molecules/CognitionSelector.css
9.   */
10. import React from "react";
11. import PropTypes from "prop-types";
12. import Slider from "../atoms/Slider";
13. import Checkbox from "../atoms/Checkbox";
14. import "../../styles/molecules/CognitionSelector.css";
15.
16. /**
17.  * @constant {string[]} ASPECTOS_COGNITIVOS
18.  * @description Lista de aspectos cognitivos predefinidos.
19.  */
20. const ASPECTOS_COGNITIVOS = [
21.   'Poca memoria', 'Niebla mental', 'Tranquilidad', 'Estrés',
22.   'Concentración', 'Distracción', 'Motivación', 'Sin motivación',
23.   'Creatividad', 'Alto rendimiento', 'Bajo rendimiento'
24. ];
25.
26. /**
27.  * @function CognitionSelector
28.  * @description Un componente que permite a los usuarios seleccionar aspectos
29.  * @param {object} props - Las propiedades del componente.
30.  * @param {Array<object>} props.cognicion - La lista de aspectos cognitivos s
31.  * @param {function} props.onChange - Función que se llama cuando la selecció
32.  * @returns {JSX.Element} El componente selector de cognición.
33.  */
34. const CognitionSelector = ({ cognicion, onChange }) => {
35.   /**
36.    * @function handleToggle
37.    * @description Añade o elimina un aspecto cognitivo de la selección.
38.    * @param {string} aspecto - El aspecto cognitivo a alternar.
39.    */
40.   const handleToggle = (aspecto) => {
41.     const existe = cognicion.find(c => c.nombre === aspecto);
42.     if (existe) {
43.       onChange(cognicion.filter(c => c.nombre !== aspecto));
44.     } else {
45.       onChange([...cognicion, { nombre: aspecto, intensidad: 5 }]);
46.     }
47.   };
48.
49.   /**
50.    * @function handleIntensidadChange
51.    * @description Cambia la intensidad de un aspecto cognitivo seleccionado
```

Home

- Namespaces
- apiConfig
- contactoEmergenciaService
- diarioService

Classes

- ErrorBoundary

Events

- SIGINT
- unhandledRejection

Global

- 404_Error_Handler
- ACTIVIDADES
- API_URL
- ASPECTOS_COGNITIVOS
- ActionCard
- ActivitySelector
- App
- AuthButtons
- AuthLayout
- Button
- CORS_Configuration
- Card
- Carousel
- Checkbox
- CircularProgress
- CognitionSelector
- DiaryBanner
- DiaryEditor
- DiaryEntry
- EMOCIONES
- EmergencyButton
- EmergencyModal
- EmotionSelector
- EmotionStats
- Footer
- GET_/
- Generic_Error_Handler
- Heading
- HeroSection
- Home
- Input
- Landing
- LandingHero
- Login
- Logo
- MainLayout
- Navbar
- ProtectedRoute
- Register
- Seguimiento
- Slider
- Text
- Textarea
- accessWithPassword
- actualizarEntradaDiario
- api
- apiFetch
- authMiddleware
- closeShareModal
- compararPassword
- componentDidCatch

```

52.     * @param {string} aspecto - El aspecto cognitivo a modificar.
53.     * @param {number} intensidad - El nuevo valor de intensidad.
54.     */
55.     const handleIntensidadChange = (aspecto, intensidad) => {
56.         onChange(cognicion.map(c =>
57.             c.nombre === aspecto ? { ...c, intensidad } : c
58.         ));
59.     };
60.
61.     /**
62.     * @function isSelected
63.     * @description Comprueba si un aspecto cognitivo está seleccionado.
64.     * @param {string} aspecto - El aspecto cognitivo a comprobar.
65.     * @returns {boolean} `true` si está seleccionado, `false` en caso contra
66.     */
67.     const isSelected = (aspecto) => cognicion.some(c => c.nombre === aspecto)
68.
69.     /**
70.     * @function getIntensidad
71.     * @description Obtiene la intensidad de un aspecto cognitivo seleccionado
72.     * @param {string} aspecto - El aspecto cognitivo.
73.     * @returns {number} La intensidad del aspecto, o 5 por defecto.
74.     */
75.     const getIntensidad = (aspecto) => cognicion.find(c => c.nombre === aspecto)
76.
77.     return (
78.         <div className="cognition-selector">
79.             <h3>Cognición y Estado Mental</h3>
80.             <p className="cognition-selector__description">
81.                 Selecciona los aspectos cognitivos que has experimentado hoy
82.             </p>
83.             <div className="cognition-selector__grid">
84.                 {ASPECTOS_COGNITIVOS.map((aspecto) => (
85.                     <div key={aspecto} className="cognition-selector__item">
86.                         <Checkbox
87.                             label={aspecto}
88.                             checked={isSelected(aspecto)}
89.                             onChange={() => handleToggle(aspecto)}
90.                         />
91.                         {isSelected(aspecto) && (
92.                             <Slider
93.                                 label="Intensidad"
94.                                 value={getIntensidad(aspecto)}
95.                                 onChange={(value) => handleIntensidadChange(a
96.                                 min={1}
97.                                 max={10}
98.                             />
99.                         )}
100.                    </div>
101.                ))}
102.            </div>
103.        </div>
104.    );
105. };
106.
107. CognitionSelector.propTypes = {
108.     /** La lista de aspectos cognitivos seleccionados. */
109.     cognicion: PropTypes.arrayOf(

```

```

connectDBAndStartServer
copyShareLink
crearEntradaDiario
create
createContacto
createRegistro
delete
deleteContacto
description
dismiss
duracion
eliminarEntradaDiario
emotion
error
formatDate
formatDateShort
getAll
getById
getContactoById
getContactos
getDerivedStateFromError
getIntensidad
getPreviewText
getProfile
getRegistroByFecha
getRegistroById
getRegistros
getRegistrosByRango
getStepLabel
goToNext
goToPrevious
handleAddActividad
handleAddContact
handleCall
handleCancelAdd
handleCancelEditor
handleChange
handleCreateEntry
handleDeleteEntry
handleEditEntry
handleEmocionToggle
handleInputChange
handleIntensidadChange
handleLogin
handleLogout
handleOverlayClick
handleRegister
handleReload
handleRemoveActividad
handleSendEmail
handleSendSms
handleShareEntry
handleSubmit
handleToggle
handleToggleExpand
handleUpdateEntry
info
intensidad
isMobileDevice
isSelected
loadContacts
loadEntries
loading
loginUser
nextStep
nombre
obtenerEntradaDiarioPorId
obtenerEntradasDiario
optionalAuthMiddleware
percentage

```

```
110.         PropTypes.shape({
111.             /** El nombre del aspecto cognitivo. */
112.             nombre: PropTypes.string.isRequired,
113.             /** La intensidad del aspecto cognitivo. */
114.             intensidad: PropTypes.number.isRequired
115.         })
116.     ).isRequired,
117.     /** Función que se llama cuando la selección cambia. */
118.     onChange: PropTypes.func.isRequired
119. };
120.
121. export default CognitionSelector;
```

port
pre-save
prevStep
promise
registerUser
render
request-interceptor
response-interceptor
sendEmergencyEmail
success
timeAgo
title
togglePasswordField
update
updateContacto
uri
useAuthStore
useToast
validateRequiredEnvVars